Seeing what's missing

Graphical and computational evaluation of imputation methodology

[TODO: create cover, add boilerplate stuff]

Acknowledgements

[TODO: write; may be added after uploading dissertation to MyPhD]

Table of contents

[TODO: fix which sections are included]

Samenvatting

[TODO: write; may be added after uploading dissertation to MyPhD]

Abbreviations and symbols

n	number of units in a sample or rows in a dataset, indexed $i = 1, \ldots, n$
p	number of variables in a sample or columns in a dataset, indexed $j = 1, \ldots, p$
Y	partially observed sample data (sized $n \times p$), also called the (hypothetically) **complete data**
R	response indicator matrix (sized $n \times p$), which stores the locations of the observed ($R_{ij} = 1$) and missing ($R_{ij} = 0$) parts of Y
Y_{obs}	**observed data** (elements of Y_{ij} where $R_{ij} = 1$), also called the **incomplete data**
Y_{mis}	**missing data** (elements of Y_{ij} where $R_{ij} = 0$)
Q	quantity of scientific interest; the scientific estimand we would like to estimate if we had complete data
$P(Q\|...)$	**complete-data model**; the analysis model we would like to fit if we had complete data, also called the 'model of scientific interest'
$P(R\|...)$	**missing data model**; the model that relates Y to R, which is interpreted as the 'probability to be missing'; also called the 'response model'
X	fully observed sample data for covariate(s) of Y, used in missing data models
ψ	parameter(s) of the missing data model that represent any cause of the missingness unrelated to X and Y
MCAR	'Missing Completely At Random'; missing data mechanism with missing data model $P(R\|\psi)$, also called 'not data dependent'
MAR	'Missing At Random'; missing data mechanism with missing data model $P(R\|Y_{\text{obs}}, X, \psi)$, also called 'seen data dependent'
MNAR	'Missing Not At Random'; missing data mechanism with missing data model $P(R\|Y_{\text{obs}}, Y_{\text{mis}}, X, \psi)$, also called 'unseen data dependent'
LWD	list-wise deletion
CCA	complete case analysis
MI	multiple imputation
JM	joint modeling; imputation technique
FCS	fully conditional specification; imputation technique
SMC-FCS	substantive model compatible fully conditional specification; imputation technique
MICE	multivariate imputation by chained equations algorithm
`mice`	multivariate imputation by chained equations software
m	number of imputations, indexed $\ell = 1, \ldots, m$
$P(Y, X, R)$	joint distribution of the incomplete data, fully observed covariate(s), and the missingness

$P(Y_{\text{mis}}\|...)$	joint (imputation) model, where $P(Y_{\text{mis}}\|Y_{\text{obs}}, R)$ denotes the posterior distribution of the missing data conditional on the observed data and the missingness
Y_j	univariate subset of Y (sized $n \times 1$); an incomplete variable
Y_{-j}	all data in Y except for column j (sized $n \times (p-1)$)
$P(Y_{\text{mis},j}\|...)$	**imputation model** for column j, where $P(Y_{\text{mis},j}\|Y_{\text{obs},j}, Y_{-j}, R)$ would indicate all other variables are imputation model predictors
$\dot{Y}_\ell$	**imputed data** in imputation number ℓ (of m total)
Y_ℓ	**completed data** in imputation number ℓ, consisting of $\{Y_{\text{obs}}, \dot{Y}_\ell\}$
$\hat{Q}_\ell$	estimate of Q calculated from completed data in imputation number ℓ
$\bar{Q}$	pooled estimate based on m imputations
T	total variance of $Q - \bar{Q}$
U	within-imputation variance due to sampling
B	between-imputation variance due to nonresponse
SE	standard error
CI	confidence interval
γ	fraction of information missing due to nonresponse, also called 'fraction of missing information' (FMI)
PMM	predictive mean matching; imputation method
V&V	verification and validation of (software) systems, where verification refers to the process of evaluating a system's output, and validation assesses its fit for purpose.

1 Introduction

TL;DR (or: the quality of the solution)

This dissertation is about algorithms and the quality of their output. I investigate how one specific type of data-generating algorithms can be evaluated for use in statistical inferences. The type of algorithms in question are imputation algorithms: algorithms with the purpose of generating plausible data values, based on a model of some observed information. This type of algorithm is popular for solving missing data problems.

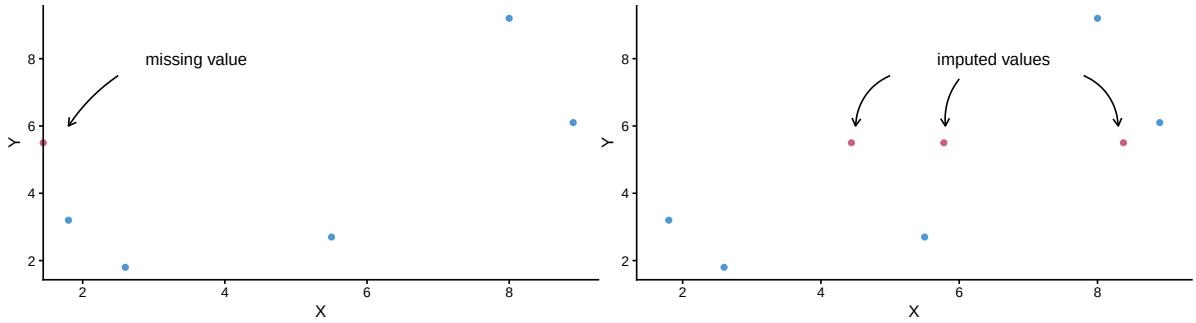
Missing data problems are timeless, but technological advances have made treating missingness easier than ever. A single line of code can seem to solve your missing data problem nowadays. But is that solution really what you expect? That is what my dissertation is about. I want to contribute methodology that facilitates drawing valid inferences from incomplete data.

Missing data are ubiquitous and cannot be ignored without risking bias. Missing data occur often in scientific disciplines with human subjects, such as the social sciences and biomedical research. A participant in a study might, for example, drop out of a longitudinal trial or refuse to answer a sensitive question, resulting in gaps in the research data. These gaps complicate data analysis and may lead to biased and invalid study results. Imputation algorithms are designed to 'impute' (i.e., fill in) missing values in incomplete datasets. Imputation is a popular and flexible procedure that separates the missing data problem from the analysis of scientific interest.

Imputation allows data analysts to first solve for the missingness, to then be able to analyze the completed data as if they were completely observed. If done several times in parallel, imputation can yield estimates that are both unbiased as well as confidence valid. The practice of drawing inference by replacing missing values with several synthetic substitutes is called 'multiple imputation' (MI). Figure 1 provides an example of multiple imputation, where a missing value is replaced several times with plausible substitutes. The variability between imputations will reflect the uncertainty due to non-response in the final estimate.

Although imputation is widely supported by software and many implementations of the method have convenient defaults, generating good imputations is not a trivial task. **How can we know when to trust the output of imputation algorithms?** There are three approaches. The ideal route is to study the formal statistical properties of the method via analytical derivations. Unfortunately, this is often impossible for imputation algorithms, many of which do not have formal proofs. Instead, the de facto standard is to estimate their empirical performance via simulation studies. These studies, however, are not standardized and are therefore hard to compare. Moreover, simulation study results might not always translate to applied use. Thirdly, for these real-world applications there is no systematic framework for evaluating the practical performance of imputation algorithms, such as diagnostics for algorithmic convergence. Ergo, evaluating whether imputations should be trusted remains challenging.

Figure 1: Multiple imputation of a missing value



In this dissertation, I set out to investigate what is needed to develop, apply, and evaluate data-generating algorithms. I thereby focus on the imputation procedure 'MICE' as implemented in the R package `mice`.

Why? Because current evaluation practices (simulation design, convergence diagnostics, visual diagnostics) are fragmented and insufficient.

How? By bridging inferential evaluation and data-/algorithm-level diagnostics for imputation methods.

1.1 Background

Why this dissertation? (or: certainty with uncertainty)

My academic tagline has long been "statistician, interdisciplinarian, open scientist". And while there's no doubt I am an interdisciplinarian and an open scientist, I am no longer keen on calling myself a statistician necessarily. I am more inclined to say I am a methodologist rather than a statistician. As a methodologist, I am interested in how science is done, and how it can be done better. Especially in the social and biomedical sciences, research should not be purely quantitative. Statistics is just *one* way of doing science.

Don't get me wrong, statistics is still a favorite of mine. I love stats (I even have a mug that says so). Statistical inference allows us to distinguish between real-world effects and random noise (Rosenberg, 2018). Epistemologically speaking, statistical inference may be classified either as deductive or inductive inference; there is no consensus in the literature about which pattern of scientific reasoning suits statistical inference (de Ruiter & Albert, 2017). Gelman & Shalizi (2013) argue that statistical inference is deductive in the context of hypothesis testing (i.e., when general laws from mathematical probability theory are used) and inductive in the context of parameter estimation (i.e., when sample statistics are extrapolated to the population). In other words, statistics is the science of uncertainty and extracting information from data (Hand,

2025). The latter can feel magical at times: the process of turning data into insights. But I am most a fan of the uncertainty aspect.

I am motivated to contribute to trust in science by making it easier to reflect uncertainty in data analysis efforts, inferences, and graphs. In this dissertation, I hope to achieve that for missing data methodology.

But first, some terminology and notation

Statistical analyses are a means to infer conclusions about a population after observing only a subset of this population: a sample. If the sample is representative, sample estimates will reflect the population characteristics plus some uncertainty surrounding it. Statisticians call this uncertainty 'errors' (e.g., the 'sampling error' due to observing only a subset of the population), which is often expressed in terms of the standard error (SE) of an estimate. Standard errors can get translated into decision tools for inference such as p-values and confidence intervals (CIs), which help researchers distinguish between signal and noise. CIs, for example, should cover the the true population parameter at the frequency of the nominal confidence level (e.g., 95%).
A statistical procedure with a coverage probability that matches the nominal confidence level is called 'confidence valid' (Neyman, 1934). Confidence validity provides an operational link between the mathematics of probability and empirical practice. Confidence-valid statistical methods give us certainty about the uncertainty in our inferences from a sample to the population. So, statistics is a means of estimating effects within a sample and inferring back to the population with some expression of uncertainty.

Uncertainty in statistical inferences can stem from many sources. In the 'Total Survey Error' framework (Deming, 1944), other sources of uncertainty are described such as non-response error. Non-response error occurs when participants do not (fully) respond to data collection efforts. We speak of unit non-response for intended study participants who do not provide any data at all (for example because they could not be reached, or refused to participate). Item non-response refers to patterns of missingness where some–but not all–data are missing for a participant (for example because they skipped a survey question). Item non-response is the focus of this dissertation: the way we handle incomplete observations in partially observed data and how that affects our inferences.

I will not pretend to provide an encompassing overview on treating missing data problems. Others have written much more extensive reference works on the matter, see for example Rubin (1996), Schafer (1997), Allison (2002), Schafer & Graham (2002), Graham (2012), Molenberghs et al. (2014), van Buuren (2018), and R. J. A. Little & Rubin (2019).

How we treat non-response is reflected in the uncertainty of our inferences. Unit non-response is often treated using weighting, which may introduce 'weighting error' Alsalti et al. (2026). It is not typically part of the TSE framework, but item non-response treatment can also produce error, such as 'imputation error' (Federal Committee on Statistical Methodology, 2001).

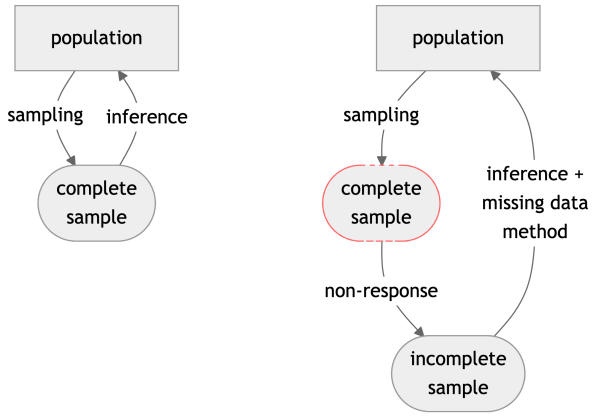
Non-response bias can be minimized by preventing missingness altogether: the best way to treat missing data is by not having any (a truism attributed to Gertrude Mary Cox). Whenever missing data cannot be prevented, the only way forward is to treat the missing data problem. Missing values force data analysts to treat the missingness in some way.

Why are missing data problematic?

Let's use an empirical example. Generally speaking, statistical inference works really well. For example, when sociologists wanted to study trust in the government during the COVID-19 pandemic, it was not feasible to survey all Dutch citizens. Instead, they used a representative sample. Some thousands of people were asked to fill in a questionnaire about their trust in the government over the course of the pandemic. By repeating the questionnaire over time, the researchers wanted to gain insight into the development of (dis)trust among different subgroups in Dutch society. The estimates from this sample would have been used to infer back to the population and inform policy, if it had not been for missing data…

Missing data are a nuisance because it is impossible to do statistical inferences with incomplete data. When there is just one missing value in a dataset, statistical inference cannot be performed. With incomplete data, even simple statistics such as the mean of variables are mathematically undefined. To obtain results from your incomplete dataset, you will always employ a missing data strategy-–whether deliberately or not.

Figure 2: Inference with incomplete data



Note. Schematic view of statistical inference in the presence of non-response. Left: no missing data in the sample from the population (i.e., ideal circumstances/business as usual). Right: incomplete sample due to non-response, so no complete sample could be obtained. What types of missing data problems are we considering? We are interested in data with incomplete cases and/or variables, or 'item non-response'. Incomplete samples where some—but not all—entries are missing per row or per column. Entirely missing rows would be 'unit non-response'; entirely missing columns would be unobserved variables.

To get an estimate from incomplete data, some kind of missing data method has to be employed. Even without specifying a missing data method explicitly, there will be handling of the missing data. The default strategy in most statistical software packages is to ignore all cases that have missing values (van Buuren, 2018). This is referred to as 'list-wise deletion' (LWD). Ignoring missingness, however, lowers statistical power and may yield invalid inferences (e.g., biased estimates, spurious significant results, see e.g., van Buuren, 2018). List-wise deletion is ill-advised under most conditions (for exceptions see e.g., van Buuren, 2018 § 2.7; Carpenter & Smuk, 2021, § 3.1). List-wise deletion results in bias because it assumes that the observed part of the data is equivalent to the missing part of the data, and this is not usually the case.

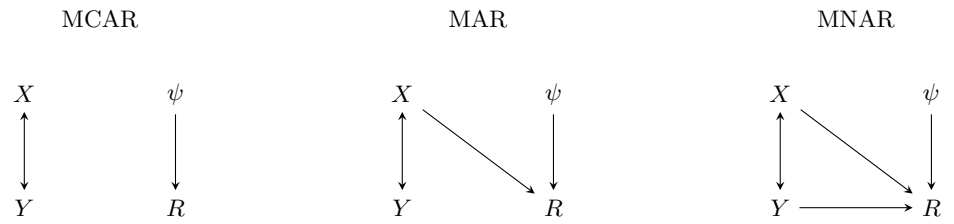
Why are the missing and observed parts not equivalent? (or: the missing data mechanisms)

List-wise deletion (complete-case analysis) assumes that the observed part of the data is representative of the missing part of the data, which seldom holds in practice. In practice, the missing part of the data is often missing due to some non-trivial reason, other than pure chance (or 'bad luck'). If there is a non-chance cause for the missingness across observations, the observed part of the data is not representative of the missing part of the data. This relationship can be formulated in terms of a missing data model. Missing data models capture the causal relation between the data (Y) and the associated missingness indicator (R). Missing data models thereby reflect the 'probability to be missing' for any given observation. Missing data models are used to describe and classify the causes of the missingness, which determines how the missingness should be handled to arrive at unbiased and confidence-valid results.

There are three missing data mechanisms described by missing data models. If there is no data-related cause cause of the missingness, the missingness is said to be 'completely at random' (MCAR), or 'not data dependent' (respective terminology by Rubin, 1976; and Hand, 2020). If there is an association between the missingness and the observed part of the data, the missingness is 'at random' (MAR; conditional on the observed data) or 'seen data dependent'. If the cause of the missingness is missing too, the missingness is 'not at random' (MNAR), or 'unseen data dependent'. Figure 3 provides a graphical representation of these three mechanisms, where the missingness (R) in incomplete data Y either depends on Y itself (i.e., MNAR), on observed covariate(s) X (i.e., MAR), or none of these (i.e., MCAR).

Modeling the non-response can aid the analysis of incomplete data. Together with complete-data models (i.e., the model of scientific interest), data models inform data analysts how the missingness might affect the scientific estimates. For example, in that study about trust in the government during the COVID-19 pandemic, it was not random who had missing data on the question about trust in the government. People with lower trust in institutions were also less likely to respond to the questionnaire. This resulted in higher missingness rates across participants with lower trust in the government. Using only the observed data to infer to the population would then leads to biased estimates (over-estimation) of trust in the government.

Figure 3: Missing data mechanisms



Note. Incomplete data Y and fully observed covariate X covary. Under missing completely at random missingness (MCAR), the missingness indicator R is only affected by ψ, causes of the missingness unrelated to X and Y. Under MAR, the missingness also depends on observed information (X), whereas MNAR entails missingness that is dependent on the missing data itself (Y). Figure adapted from Schafer & Graham.

Methods that implicitly assume MCAR (like list-wise deletion) are therefore likely to cause bias.

In human subjects research, pure MCAR missingness is unlikely. Only truly random missingness, independent of any individual characteristics, would amount to missing completely at random missingness mechanism. This could happen, for example, due to a glitch in the service software, affecting all participant equally likely. In our case, however, there were clear signs of non-MCAR missingness. If we can model the missingness probability based on known data, such as previous observations or demographic data, we have 'Missing At Random'. Then, conditional on this information, observed and missing values have the same probability to be missing. Under MAR, we can model the missingness probability using known variables and obtain valid inference if this model is adequate. This is in contrast to the 'unseen data dependent' missingness mechanism or 'missing not at random'. If the missingness depended the unobserved values themselves, we speak of 'Missing Not At Random'. For example, when people who distrust the government are less likely to respond to the trust item, and we have no way of correcting for this. This kind of missingness cannot be resolved from the observed data alone and requires assumptions or external information. In practice, it is impossible to know from the observed part of the data alone whether we are dealing with MNAR, so you always have to make an assumption about the mechanism underlying the missingness in your data.

If the missingness is MNAR, the validity of the inferences is at risk. MNAR missingness is often called 'non-ignorable', a term which roughly means that given the observed data, you cannot treat the missing-data mechanism as irrelevant for the subsequent inferences. This is in contrast to 'ignorable' missing-data mechanisms, which meet two requirements: 1) conditional on the observed data, the probability that an entry is missing does not depend on the unobserved value itself (i.e., MCAR or MAR missingness), and 2) the parameters in the missing data model are distinct from the parameters of the substantive model (Little & Rubin, 2020). If treated correctly, ignorable missingness will yield valid inferences.

How to treat the missing data problem? (or: ignore, model, fill in – the missing data methods)

This section describes three popular routes for treating missing data. I will not pretend to provide an encompassing overview on treating missing data problems. Others have written much more extensive reference works on the matter, see for example Rubin (1996), Schafer (1997), Allison (2002), Schafer & Graham (2002), Graham (2012), Molenberghs et al. (2014), van Buuren (2018), and R. J. A. Little & Rubin (2019). The three missing data treatments I will describe are deletion-based methods, likelihood-based methods, and imputation.

The first missing data treatments are deletion-based methods, such as list-wise deletion. With list-wise deletion, we only analyze rows in the data without missing values (i.e., complete cases). Deletion-based methods are easy to apply, and sometimes even applied by default in statistical analysis software. If the missingness level is low, deletion-based methods do little harm, and under some conditions they might even be the best missing data method to apply (see e.g., van Buuren, 2018 § 2.7; Carpenter & Smuk, 2021, § 3.1). Deletion-based methods, however, do not generally accommodate differences between the observed and missing parts of the data, making them mostly applicable to MCAR. Under most kinds of missingness, deletion-based methods will therefore risk biasing the statistical inferences. Moreover, these methods discard incomplete observations, resulting in lower power and potential resource waste compared to non-deletion-based methods. In short, deletion-based methods do not treat the missing data as much as ignore them, and one should be careful of bias when applying them.

Secondly, there are methods that incorporate the missingness into the model of scientific interest. Modeling methods do accommodate MAR, but are often quite complex. These missing data methods jointly specify the complete-data model and missing data model per analysis. For example, Bayesian methods can incorporate the missingness into the analysis model. But you have to specify the model for each analysis separately, and there are no null hypothesis significance tests which is a problem when working with applied researchers. Another option to incorporate the missingness in the analysis model is expectation maximization. But this yields many parameters and might not be available for a complex multilevel model like the COVID-19 data. Both of these techniques are tied to a specific analysis model, which might not be suitable for contemporary practitioners who run several models in a data science workflow.

Thirdly, there are imputation methods. Imputation methods do not have these drawbacks of being computationally advanced and analysis-model specific. Imputation splits the missing data problem from the analysis of scientific interest. Imputation methods first solve for the missingness, to then analyze the completed data 'as if' they were fully observed. This allows the use of familiar complete-data methods instead of bespoke incomplete-data models. Imputation is useful when different people are responsible for different parts of a project (e.g., a missing data methodologist vs. domain expert). And when many different analyses are planned on the same data, it is possible to run several analyses without having to incorporate the missingness into the scientific model each time. This makes imputation better suited for contemporary data science workflows than missing data methods that incorporate the missingness in the

substantive model. Compared to missing data methods that incorporate the missing data like Bayesian methods or likelihood-based approaches (e.g., EM), imputation is more flexible and often less complex for non-experts. Compared to list-wise deletion, imputation methods offer the ability to accommodate MAR mechanisms, and they reduce research waste by retaining incomplete cases. Imputation thus has the ability to produce unbiased estimates. But are they properly uncertain (i.e., confidence-valid) too? That's where multiple imputation (MI) comes in!

Why multiply impute? (or: several 'wrongs' *can* make a 'right')

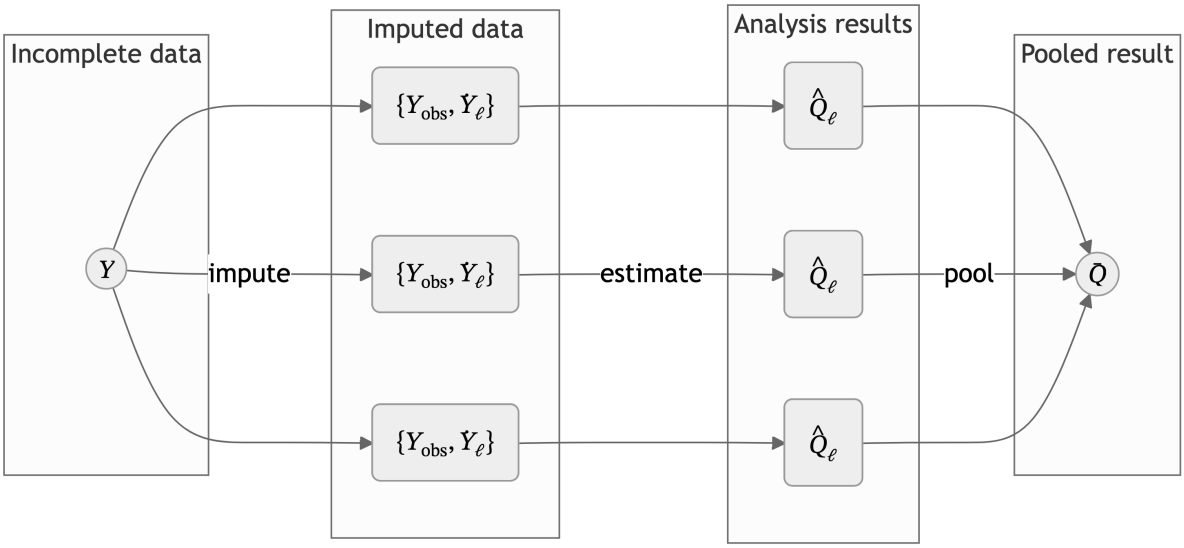
All techniques that rely on a single replacement value risk under-representing uncertainty in the inferences, because the uncertainty due to the non-response is not taken into account. The uncertainty due to non-response can be captured using the multiple imputation framework. Conceptualized by Rubin (1976), MI posits that repeated stochastic draws per per missing datapoint create a distribution of plausible values: the posterior distribution of the missing data conditional on the observed data and the missingness, $P(Y_{\text{mis}}|Y_{\text{obs}}, R)$. This posterior distribution reflects the uncertainty due to non-response. By drawing imputations from the posterior distribution of the missing data, we obtain completed versions of the data that *might have* been.

MI is the practice of generating several complet*ed* datasets by replacing each missing value with a synthetic substitute multiple times. The completed datasets can be analyzed as if the data were complete to begin with. On each completed dataset, we obtain estimates of the analysis of scientific interest. These estimates per imputation are then combined to obtain one pooled estimate. The variance of the estimate incorporates not just the sampling variance (within each imputation) but also added variance due to non-response (between imputation variance and simulation error). The variability between imputations reflects uncertainty due to the missingness.

The result is a probability distribution for each missing value, from which we subsequently draw several values as imputations. If we would use infinitely many values as imputations, we would obtain the entire probability distribution. In practice, however, we use a limited number of imputations. With that, we approximate the probability distributions of missing values. The more imputations are used, the better we can represent the uncertainty in the data due to missingness. This is expressed in the 'simulation error'.

Multiple imputation means that each missing value is filled in several times, with slightly different model parameters to represent model uncertainty. The result of MI is that the uncertainty in the data due to missingness is preserved in the inference. Correcting for missingness happens at two levels: 1) modeling the relation between missing cases and covariates; 2) adding the appropriate variance to the models to represent the uncertainty due to missingness. Under the right conditions, MI yields valid, unbiased estimates (Schafer, 1997). MI enables researchers with missing data problems to correct for differences due to missingness and maintain the

Figure 4: Multiple imputation scheme ($m = 3$)



Note. Read left to right. Incomplete data Y is partially observed, $Y = \{Y_{\text{obs}}, Y_{\text{mis}}\}$. The missing data Y_{mis} is imputed m times to obtain m completed datasets $\{Y_{\text{obs}}, \dot{Y}_\ell\}$, where $\ell = 1, 2, \ldots, m$. On each imputation, the quantity of scientific interest Q is estimated, yielding m analysis results $\hat{Q}_\ell$. These results are pooled across imputations to obtain a single pooled result $\bar{Q}$.

appropriate level of uncertainty. The aim is to generate imputations to fill in the missing part of the data. How do we model the missing part of the data based on the observed part of the data?

1.2 Imputation in practice

How to generate imputations? (or: univariate, joint, chained – the imputation flavors)

We model the distribution of this missing part based on what is observed, using the observed part of the data to fix the missing part of the data. Which types of model or models do we use?

The simplest model is no model at all. With hot deck imputation, we use random other person's observation to impute or fill in the values for people with a missing data point. The problem is that this disrupts the multivariate distribution of the data, so we get biased estimates. Another simple technique is to impute a univariate statistic such as the mean. Unfortunately, imputing the mean will bias your scientific estimates even worse than list-wise deletion would, because mean imputation severely biases any multivariate statistics downward, and there is no correction for the missing at random missingness mechanisms. In general, all univariate imputation methods fail to retain multivariate associations in the data, and therefore cannot account for MAR mechanisms. So, instead of imputing variables univariately, imputations should retain the multivariate structure of the data.

Capturing the multivariate structure of incomplete data is historically done in two steps: a modeling step and an imputation step. In the modeling step, a multivariate distribution of the full dataset would be defined, after which imputations could be drawn from the joint model. In a joint model, one specifies $P(Y, X, R)$: the distribution of the incomplete data, complete covariates, and missingness indicator. Joint modeling imputation was originally developed for known distributions, such as multivariate normal data. With increasing data complexity and dataset size, specifying a joint model might not be feasible. The problem with joint modeling is that there might not exist a multivariate probability distribution, for example, with factor variables involved. Joint modeling is a direct and inflexible way to arrive at a joint distribution. Good thing we do not need this joint model explicitly.

Missing observations can be filled in using one or more imputation models instead. While Rubin (1987, pp. 160–166) "distinguished three tasks for creating imputations under an explicit model: the *modeling* task the *imputation* task and the *estimation* task." (van Buuren, 2007, p. 221; in Molenberghs et al., 2014), FSC does not require these explicit tasks. In the FCS framework (van Buuren et al., 2006), we only need an imputation model for each incomplete variable. Imputation models consist of the functional form of the model (e.g., stochastic regression) and imputation model predictors (i.e., other variables in the data). FCS is substantive model agnostic, and does not assume any specific joint distribution. Instead, FCS only requires an *imputation model* to describe how synthetic values may be generated, without explicit

specification of the joint model. This makes FCS flexible. Just specify an imputation model per variable.

How to define imputation models?

We define an imputation model per incomplete variable. And we need two things for this: 1) We need to specify the functional form of the model. For example a semi parametric method such as predictive mean matching for Likert scale items such as trust in the government. And 2) we need to choose imputation model predictors. For example, variables with a high linear association with this incomplete variable, or variables that are related to the missingness indicator of this incomplete variable (such as demographic data).

Some things to keep in mind: With FCS, the quality of the imputation model can only be determined/assessed based on the complete data model. But what if that model is not (yet) available? Then take the most complex possible model and check that, because if you would take the simplest model, then that would implicitly mean you are constraining parameters (forcing b=0), systematically suppressing relations. Don't forget to add the effect of interest in your model of scientific interest. Otherwise, you are constraining the relation between the variables to be zero. If there is a specific data analysis model for the effect of scientific interest that you already have in mind, consider substantive model compatible fully conditional specification. SMC-FCS (Bartlett et al., 2015) is optimized for known analysis model models: we assume a complete data model and derive the imputations from there. But what if the analysis model is unknown? Well, Fully conditional specification is the substantive model agnostic. So we build an imputation model based on the most complex analysis model that we might want to test.

FCS implicitly relies on the joint distribution. The nice thing about joint modeling is that convergence of the model is guaranteed, whereas this is not the case with fully conditional specification. "Imputation models bypass the need to specify $P(Y, X, R)$, though their use creates new responsibilities for substantiating its correctness for a given statistical analysis." (van Buuren, 2007, p. 222). The question becomes: how do we choose an appropriate imputation model? And how do we know whether we have chosen an appropriate imputation model?

FCS has no deductive or analytical proof, so we have to rely on evidence from simulation studies. It has been shown that FCS works under many conditions. But how to know it works in practice? We need assumptions. Many modeling assumptions inherent to a model-based technique like MI cannot be tested without the missing data itself. So, like assumptions for GLM, we look at the data to evaluate the appropriateness of our chosen model. Similarly, we need to evaluate the choice of imputation models for signs of assumption violations. Some assumption checks are visual inspection, some are quantitative, some need external info.

How to evaluate imputation models?

How do methodologists know whether an imputation method works? Because we have empirical estimates from simulation studies (and, if available, formal properties from analytical derivations). How do imputers know whether their imputation models worked? Our current answer is well, they don't know for sure. And that's what this dissertation is all about.

Life cycle of imputation methods: method is theorized, method is implemented in software, method is evaluated in simulation study, method is put to scrutiny by scientific community, method is applied in practice, method is evaluated in real-world data.

There are three main routes to evaluate imputation models. One can evaluate:

- the formal statistical properties of the method via analytical derivations;
- the empirical performance (estimates of the method's attributes) via simulation studies; and
- the operational utility (practical usefulness) in real data.

The first and preferred option is to obtain the formal statistical properties of the imputation method using analytic derivations. Often this is impossible or unfeasible with fully conditional specification, when no known joint model exists. Without this, we cannot definitively conclude that an imputation method is an unbiased estimator. Instead, we estimate the empirical performance of these methods using simulation studies. This is the default evaluation in the missing data methodology field.

Simulation studies are the standard, but not standardized. If we have a specific analysis model of interest, we can calculate performance metrics such as bias and confidence interval coverage to conclude whether a method is unbiased and confidence valid for a certain estimand. This requires a specific analysis model, which might not be the case. And it is impossible to cover all the potential use cases. Another problem is that this is not standardized, so it's difficult to compare different simulation studies. We could also calculate the imputation accuracy by comparing the imputations against the complete data, but this is ill-advised.

The third option is the operational utility. This is case-by-case evaluation in applied use with real data. In practice, confidence validity is not guaranteed. Multiple imputation aims to propagate uncertainty due to missingness, but only under specific conditions such as correct assumptions about the missingness mechanism, 'congeniality' between imputation and analysis models, and adequate algorithmic behavior. When these conditions fail (e.g. under MNAR missingness, incomplete imputation models, or non-convergence in the algorithm), the estimates may not be unbiased and/or confidence-valid. Therefore, we check the imputation model itself, for example to see whether the algorithm has converged. We also inspect the output and we might run sensitivity analyses. This is not systematic at all currently. That is a problem. Users of the method then do not know whether they have correctly applied the method.

In this dissertation, I focus on the second and third routes. We will borrow some terms from related fields. I will connect it to concepts from systems and software engineering, and the information retrieval sub-field of computer science.

1.3 Evaluation tools

Computational evaluation, verification and validation [TODO: edit]

From a computer science perspective, imputation workflows are 'sociotechnical systems'. In contrast to a purely technical system, a sociotechnical system is a collection of interrelated components that delivers services to users, consisting of software, hardware, *and* human operators (Sommerville, 2015, p. 556). Sociotechnical systems "include one or more technical systems but, crucially, also people" (Sommerville, 2015, p. 552), and their success relies on the joint functioning of the system components (Sommerville, 2015, p. 556). Sociotechnical systems have three characteristics, namely:

- emergent properties of the system as a whole, which only becomes apparent when integrating the separate components (e.g., dependability);
- non-deterministic outputs, resulting from human operators who may differ in the choices they make; and
- subjective success criteria, with objectives that depend on the context of evaluation (Sommerville, 2015, p. 558).

When we apply this terminology to imputation workflows, we can conclude that `mice` is a technical system: a piece of software with algorithms to generate imputations. Applications of `mice`, however, are sociotechnical. In practice, imputation workflows include `mice`, the imputation script, the analysis script, visualizations, and the researcher interpreting the output and deciding what to accept. Imputing is a sociotechnical system, which includes more components than just `mice`. Successful use of `mice` depends on the usability of the software in practice.

Since "Systems have properties that only become apparent when their components are integrated and operate together.", we should evaluate the imputation workflow not as separate functions, but as a whole. For imputation methods this means that they should work in practice, not just in simulations. For such systems, analytic guarantees (e.g., asymptotic unbiasedness of MI) are not enough. We need computational and empirical checks on the whole workflow. And there should be interoperability with other systems an imputer might use in their imputation workflow, such as `ggplot2` for data visualization, and statistical analysis packages such as `stats`. That is where V&V comes in.

Verification and validation (V&V) is a process for the evaluation of systems, used when the 'high road' of formal proofs is blocked. It is framework for checking whether we have both built the system correctly as well as built the correct system.

In systems engineering, the system life cycle framework is used to build, develop and evaluate systems. There are standards for the evaluation of a system: verification & validation. V&V is needed when the 'high road' of analytical derivations and proofs are impossible or unfeasible.

Verification asks 'was the system built right?', whereas validation asks 'was the right system built?'. So with verification, we can check the quality of a system by plugging in an input with a known output. Do we get the correct output in return? For imputation methodology, this is what we do with simulation studies and with unit tests in software. We want to check whether the system (or the method in this case) conforms to the requirements that we set (e.g. accuracy). Verification is about conformity to specifications, usually before or independent of real-world use.

Then validation, on the other hand, asks whether the system is suitable for operation by the actual users in a specific intended use or application. Is a system able to accomplish its intended use? Validation = use of use cases and usability testing to evaluate whether users have all they need to use the system. In stats, it's proving that a statistical model or analytical procedure behaves as intended under real-world operating conditions. Validation is often sociotechnical: it combines technical inspection with *human judgment* and domain knowledge. Validation test = "test to determine whether an implemented system fulfils [sic.] its specified requirements" [ISO/IEC 2382:2015, Information technology — Vocabulary] For imputation methodology, it is tools for users to evaluate their imputation workflow with specific data (and, optionally, their intended analysis models). This dissertation describes computational and graphical evaluation tools. In the future, we might use integration tests and user evaluation studies too?

What is computational evaluation and diagnostic visualization? Computational evaluation encompasses a range of systematic, computer-based evaluation tools such as numerical experiments or simulation studies. Computational evaluation = use of metrics, tests, and simulations to evaluate properties of the system with minimal human judgment. Diagnostic visualization is the use of graphs to communicate system behavior to stakeholders. It's 'diagnostic' because the purpose is to identify patterns and problems instead of descriptive/summaries. Diagnostic visualization/visual inspection helps stakeholders understand the system's behavior and judge whether it's acceptable for their needs.

What is computational evaluation? [TODO: edit]

Computational evaluation is originally from information retrieval research (see e.g. "Evaluation Measures (Information Retrieval)," 2025) The practice of computational evaluation combines *effectiveness* (i.e., did the process yield the correct result, e.g., accuracy, precision, recall, etc.), *system quality* (e.g., speed, coverage of all possible results, etc.) and, importantly, *user utility* (e.g., happiness, productivity, etc.) (Manning et al., 2008) The term 'computational evaluation' is not specifically defined for the field of statistics, but it could be useful. One definition of computational evaluation is "to determine the quality of solutions attainable" (Dudek, 2004) Translated to statistics, a model may be statistically valid, but not fit for purpose. For example,

computations that take too long to converge for use in a production pipeline/real-time use in patient care. Another definition of computational evaluation is "the process of ensuring an algorithmic solution is a good one: that it is fit for purpose" (Curzon et al., 2014). We can use this when a statistical process has different purposes and thus different usability/requirements. For example, calculating uncertainty at the level of a single case, a sample, or inferring to a population. Since the term is not defined for statistics, there is no definition to use for the computational evaluation of imputation methodology. In this dissertation, I want to plant a flag: how can we define 'computational evaluation' in the context of imputation methodology? Working definition: *systematic assessment of the algorithmic behavior and data-level outputs of imputation procedures*, focused on: algorithmic stability (convergence, failure modes), sensitivity to modeling choices, plausibility and coherence of imputed values, and usability for applied researchers. These aspects are often already taken into account by imputers, but I try to systematize and operationalize these aspects that are often informal or *ad hoc*.

Computational evaluation is not equal to statistical evaluation. Computational evaluation goes beyond statistical evaluation. Statistical evaluation is important, e.g. to quantify bias and variance of estimators/procedures, but not the focus of this dissertation. Computational evaluation is something to check *conditional on* acceptable statistical properties, and does not replace statistical evaluation. Dimensions of computational evaluation: e.g. inferential validity, algorithmic behavior, data-level plausibility, user utility, …

Why should we care about computational evaluation of imputation methodology? [TODO: edit]

What do we care about when evaluating imputation workflows? Statistical properties such as bias/coverage/variance, empirical relevance/plausibility of the imputations, and software-engineering nitpicks like speed/convergence/fault rate. Statistical evaluation is not possible in practice, only in simulation studies. Why is statistical evaluation may not possible IRL? Bias and confidence interval coverage are impossible to determine without a comparative truth (although PPC and bootstrapping allows us to compare with respect to observed values). In this dissertation I will use computational evaluation as a suite of tools to investigate the domains of algorithmic stability, data plausibility, and user burden, next to inferential validity.

User utility does not only require statistical validity, but also usability/appropriateness/plausibility of the imputations. Even if we know the process yields correct inferences, we can evaluate the 'fit' of the solution.

One question to consider is: do the imputed values lie within the range of the observed values? For statistical validity this is not needed. Imputation theory does not require the imputed values to be plausible. Rubin (1976, 1987) developed the methodology to get correct population-level inferences from incomplete samples, not at the individual case-level within the sample. [TODO: consider adding that since then, data analysis has changed] Bluntly stated: imputation pragmatists/purists don't care for the imputed values themselves, as long as they

result in statistically valid inferences. But end-users (e.g. clinicians) do care for plausibility. For example, imputing negative values for age will yield 'unrealistic' imputed values. Sometimes this is bad, such as using PMM for a variable with a sparse region, because PMM with part of sample space with too little donors might lead to bias. Sometimes this is okay, such as with polynomials (ignoring the U in imps *could* still get statistical validity).

What are other limitations of purely inferential metrics (bias, coverage)? Statistical evaluation relies on known analysis models, while FCS is substantive model agnostic. So, supplement it with computational evaluation and visual inspection. These are also possible without analysis model, which makes it more pressing today, because imputations might be used with several analyses. Not knowing the analysis model up front risks non-congeniality

Another problem of unknown applications with potential non-congeniality is non-convergence. Without graphical or computational diagnostic for non-convergence, users might analyze imputed values of non-converged chain. For example, fcs iterations not stable, but estimand okay (because in simulation study we have true value), but what about conv at cell level? Statistical models like MI are optimized to distinguish between effect and noise of population averages–not the individuals that make up your sample.

But also vice versa: when might the plausibility of imputed values mislead a data analyst into believing the imputations are statistically valid? When 'looking reasonable' is confused with 'being consistent with the data-generating process', such as single imputation (stoch reg), or imputing under MCAR assumption when there is a MAR mechanism, or destroying the joint distribution of the data while preserving the marginals. Another example: psych study with baseline, some of which are missing; baseline, intervention, follow-up –> mice would inflate baseline and follow-up if both are incomplete; statistical evaluation will not show this (unbiased, cov ok, etc.); over-imputation or postpred check would show diff distr for obs and imp; circularity should be removed: follow-up should not predict baseline So computational evaluation should never replace statistical evaluation.

1.4 Evaluation of imputation

Building and evaluating imputation models (or: human in the loop)

Imputation with `mice` is a sociotechnical process. The software will happily produce imputations with its defaults, but *good* imputation requires human input. There are some decisions that you, as imputer, implicitly or explicitly make. Figure 5 shows where human input in requested in the imputation workflow.

A data analyst faced with incomplete data should not start imputing right away. They should first ask themselves: what kind of missingness am I dealing with? Inspect the patterns of misisngness and consider which missingness mechanism(s) might be at play. A missign data pattern plot tells you where missingness occurs (which variables, which cases) and which variables tend to be missing together. For conclusions about the misisng data mechanism, we

need external information. Imputation with `mice` assumes ignorable misisngness. MCAR and MAR are modeling assumptions, not empirically verifiable facts. In practice, it it impossible to determinately differentiate between MCAR and MNAR. But specific MAR mechanisms may become apparent through computational and graphical evaluation (e.g., by calculating or plotting associations between observed data and missingness indicators). If MNAR is suspect, use dedicated tools and/or design sensitivity analyses to assess how robust your conclusions are. If MAR mechanisms show up, include the relevant relations into the imputation models.

For each incomplete variable, `mice` asks you to specify an imputation method. This is a choice of functional form of the imputation model (e.g., logistic regression for dichotomous variables). How do we know which functional form to choose? The simplest option is to accept `mice`'s defaults, which are motivated by simulation studies and pragmatism. For example, the default method for continuous variables is the semi-parametric method `pmm`: predictive mean matching. With `pmm`, observed data entries are used as donors when imputing missing entries, conveniently yielding imputed values that lie within the range of the observed data. Diverting from these defaults might be useful in case of big, wide, or sparse data. That is, some methods can be computationally inefficient with big data (large n, large p), mathematically undefined with wide data ($n < p$), or yield suboptimal imputations because of a lack of donors in sparse data ranges. Moreover, if the column type in the incomplete data is not correctly specified, `mice` will assign incorrect imputation methods. Therefore, inspecting the distribution of each incomplete variable is advised (REF?). You can use bar charts, histograms, and density plots of the observed part of the variable to assess the distribution of the incomplete variables and determine imputation methods. Visual inspection can also be used to check whether there is skew or spikes (e.g., structural zeros) that require transformations. In short, the imputation method determines the distribution of the imputed values, which should match the distribution of the observed values to at least some extent.

The next major decision concerns the predictor matrix: which variables should be used to impute which? Some common sources of input are the associations assumed by the analysis model, associations in the observed data, associations with missingness indicators, and external input and design information. If you already know your primary analysis model (e.g., multivariate regression), it is common to include all analysis model predictors and outcomes in the imputation models. Otherwise, that would implicitly constrain these parameters to zero in the imputations, systematically suppressing relations (i.e., biasing the regression coefficients in the analysis model towards zero). In many modern workflows, the imputed data will be used for multiple, possibly unknown, analyses. You may therefore include a broader set of predictors than strictly needed for one model, or adopting 'auxiliary variables' (i.e., extra imputation model predictors that are not part of the analysis model or variables of interest). You may include predictors that are strongly associated with the incomplete variable, based on corralations ("include an auxiliary variable if its correlation with the main variable is 0.5," Nguyen et al., 2021, p. 4) Not just associations with each incomplete variable may be used to select imputation model predictors. We can also use the association of the missingness indicator with other variables in the data. Predictors that explain missingness can help you approximate MAR or correct for design-related patterns. Other design-related patterns require expert knowledge e.g., about

branching or which covariates should be included to mitigate MNAR. Choosing imputation model predictors encodes your beliefs about what information is relevant for reconstructing missing values and for capturing missingness mechanisms.

After specifying methods and predictors, you still need to ask whether the imputation model behaves adequately. We need to evaluate the imputation model post hoc. Some evaluation tools include algorithm-level diagnostics (e.g., traceplots of key parameters over iterations to assess algorithmic convergence), visual inspection of the distribution of the imputed values (e.g., to identify mismatch between observed and imputed data distributions), and sensitivity analyses (e.g., via over-imputation, which temporarily treats some observed values as missing to compare imputations to the true values).

These evaluations are neither fully automated nor purely objective. They involve building and reading plots, interpreting diagnostics, and deciding what degree of misfit is tolerable for your research aims.

Apart from the imputation models, other choices also affect the quality of the imputation workflow and resulting statistical inferences. For example, the number of imputations (m, or `m` in `mice`) and number of iterations in the imputation algorithm (`maxit`) may interact with the visual and computational diagnostics above.

Taken together, these decisions show that a `mice` workflow is not a push-button solution. It is a modeling exercise in which the user is involved in model building and evaluation. Human input makes imputation workflows subjective to individual choices and preferences.

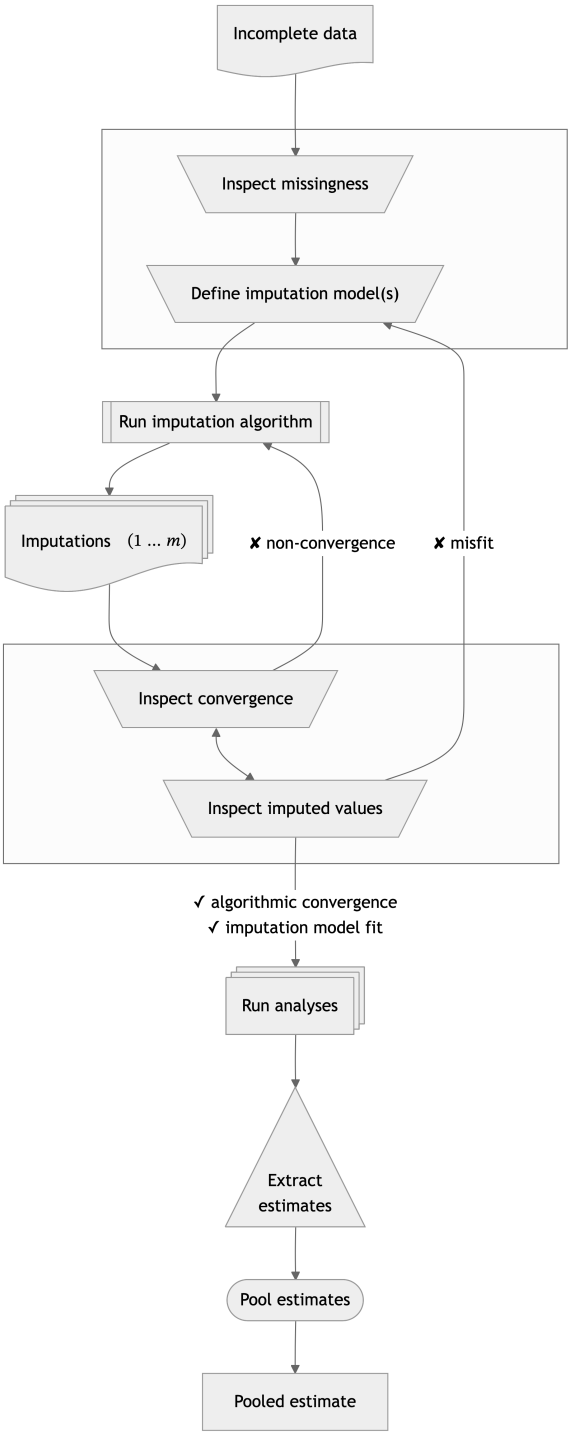
What is the scope of this dissertation? [TODO: edit]

In this dissertation, I will focus on missing data problems with item non-response and MCAR/MAR missingness mechanisms. You might expect me to go into…

- In the category non-response: just item non-response (not unit non-response), and M(C)AR mechanisms (not MNAR). Unit non-response and MNAR missingness mechanisms are outside of the scope of this dissertation.
- In the category imputation algorithms: just FCS in the flavor of MICE (not JOMO or SMC-FCS). The missing data method under consideration is primarily multiple imputation by chained equations (MICE), but some results may generalize to other iterative (FCS) imputation algorithms. We do not cover joint modeling (JOMO) imputation or non-imputation missing data methods. Although MICE is not SMC-FCS, we will restrict the analysis of scientific interest mostly to GLM family of models.
- In the category substantive models: just GLM for statistical inference. The applications are aimed at statistical inference (not prediction or causal inference).

RQ: what tools are required to assess the quality of data-generating algorithms? in the development, evaluation, and application.

Figure 5: Multiple imputation flowchart



Note. Statistical evaluation is mainly focused on the relation between the incomplete data and pooled estimates. Computational evaluation is in every trapezoid: processes that require human intervention. In the first box, the question is: 'how can we model the missingness?', in the second box, the question is: 'do the imputation models fit the data?'. Missing in the graph: drawing inference with pooled estimate, and use external info after inspecting the missingness to involve all stakeholders (CRISP-DM steps).

What is the status quo? Verification via statistical evaluation (but not standardized), and some validation via computational and graphical evaluation (but not systematic). Is the status quo sufficient? No, statistical evaluation is required, but not sufficient. How Rubin developed the methodology works in theory, but is not enough for contemporary practitioners.

Can we use existing tools or do we need more? and how? No, not all tools are present in the imputer's toolbox. I contribute to the new status quo ("extending" = generalizable scientific contribution of this dissertation) by unifying how imputations are evaluated across the life cycle. There two phases: statistical properties of the method in simulation (e.g., bias and confidence-validity; statistical evaluation) & practical application (graphical and computational evaluation).

Gold standard: analytical properties. Deductive truths. But not feasible with FCS. So, implementation needed, which does require V&V.

Methodologists need standardized computational evaluations to justify new procedures, while applied researchers need accessible tools to assess whether a chosen method behaves adequately on their particular data.

- Verification of statistical properties via formal analysis (e.g. proofs) and/or simulation
- Validation of operational utility via graphical and computational evaluation

Verification

Simulation studies are verification. Assessing whether the system yields correct (known) output when a specific (known) input is supplied. Statistical evaluation is the default to evaluate the inferential validity. Examples are unit tests for software and performance assessment of methods via simulation. Verification via simulation is done, but not standardized. Verification via testing is mostly done as unit tests, but not as integration tests.

Validation

In simulation studies we can construct the "truth", but in practice we have to rely on assumptions. Those assumptions are crucial precisely because they cannot be (fully) proven. In practice, you cannot validate whether the imputation model was appropriate, because what you would need to do so is exactly the thing you don't have: the missing part of the incomplete data, while you only have the observed part. The thing we need is not available by definition. In practice, the best thing we can do is to rely on a combination of assumptions/theoretical justifications/substantive expertise and indirect evidence of imputation model fit: check the observed versus imputed data distributions, and inspect the imputations for signs of non-convergence. Some assumptions are impossible to definitively determine (e.g. MNAR), some we can evaluate by looking at the data. Inferential validity cannot be established.

Computational (quantitative) evaluation – metrics, tests, simulations. Graphical (qualitative) evaluation – diagnostic visualization/visual inspection. This dissertation offers insight and tools to do so. Validation tools to assess operational utility.

Chapters:

1. Evaluation paper: evaluating (new) imputation methods via simulation studies/how we know FCS works
2. Convergence paper: evaluating imputation algorithm quantitatively
3. Software: choosing/evaluating imputation models in practice

Introducing chapter 2: evaluation

This chapter argues that imputation methods should be subjected to a standardized evaluation framework for siumulation studies. How to design simulation studies that are comparable? Which simulation conditions to consider, which performance metrics to report? And how to document this in such a way that others can reproduce and reuse the evaluation?

The setting where we do have the comparative truth (the true but missing values) is simulation studies. Imputers have to rely on simulation study results in order to choose the best imputation method for their incomplete variables. That's why simulation studies for imputation methodology should be comparable with one another. Then, imputers can make a 'grounded' assessment which imputation methods may be applicable, and choose the most appropriate one. Example: NRI search for lower bound, so impute as 'positive behavior', but per definition wrong statistical inf, because we intentionally bias in one direction

This chapter asks how we should judge, in a systematic way, whether imputation strategies work. **RQ: what to look out for when designing simulation studies for the evaluation of imputation methodology?**/How can computational evaluation criteria be systematically incorporated into simulation studies for imputation methodology?

- How can we trust simulation studies for imputation methods?
- How can we make them compatible & fair?

Focus on methodologists (who design imputation methods and study their properties) as opposed to imputers (applied researchers who apply imputation models to specific datasets). Methodologists need comparable, reproducible evaluations of imputation methods across scenarios. Without a shared evaluation framework, claims about 'valid, unbiased estimates' for MI might not translate to practice. How can we standardize the verification of imputation methodology?

The chapter offers advice for designing simulation studies (i.e., computational evaluation) based on literature review Liu et al. (2023).

Introducing chapter 3: convergence [TODO: add]

This chapter investigates non-convergence in the MICE algorithm. FCS imputation, as implemented in `mice`, is an iterative procedure. Incomplete variables are imputed one-by-one, conditional on other variables. If these other variables are incomplete too, iteration is required. After the first cycle through the columns, we can get better imputations based on the now-updated imputation model predictors. In `mice`, this cycle is several times until convergence is reached (by default five iterations). Because of its iterative nature, convergence *should* be a requirement for valid inference, or is it?

This chapter investigates the questions: **How does non-convergence in iterative imputation affect inferential validity? And how can non-convergence be diagnosed?** The question is answered via a simulation study.

Introducing chapter 4: `ggmice`

This chapter introduces the software `ggmice`: an R package for building and evaluating imputation models. The `ggmice` package offers tools for the visualization of incomplete data, imputation models, and imputed data. `ggmice` supports `mice` workflows by making imputation modeling choices visible and, therefore, open to scrutiny. Compatibility with `ggplot2`'s grammar of graphics makes `ggmice` graphs extendable with the familiar tools of the `tidyverse`, and adaptable for publication-ready visualizations.

Indicators for adoption: how is the software already used?

Although `ggmice` has not undergone the traditional journal-based peer review, it has been vetted in practice. The package has been released on CRAN in 2022 (Oberman et al., 2025). CRAN downloads are reaching approximately 1.000x per month at the time of writing. The accompanying Zenodo publication (Oberman, 2022) has been cited in seven scientific works. Open source code and issue tracking are available on GitHub (github.com/amices/ggmice). There have been multiple external 'Issues' and 'Pull Requests' on GitHub, indicating that other researchers and developers have actively reviewed and improved the codebase. In short, there is active use of and involvement in `ggmice`, as indicated by regular downloads from CRAN, citations to the Zenodo record in the scientific literature, and community feedback via platforms such as GitHub and StackOverflow.

Design principles

`ggmice` is developed according to 'open source' and FAIR principles. The package is implemented in the open source language R, distributed via CRAN and GitHub, and archived on Zenodo. Its evolution is driven by user feedback (e.g., GitHub issues, StackOverflow questions, conference

discussions, teaching) and collaboration (e.g., code contributions, statistical consulting with external partners).

Other design principles are grounded in `ggmice`'s compatibility with the popular R packages `ggplot2` and `mice`. `ggplot2` offers flexible data visualizations based on the 'grammar of graphics' framework. `mice` offers flexible imputation workflows for inference with incomplete data. Together, these packages inform the design and functionalities of `ggmice`.

`ggmice` produces `ggplot` objects, leveraging the 'grammar of graphics' to create layered, easily adjustable plots. The figure that appears is the output of a series of design decisions. The grammar of graphics treats plots as structured ways of reasoning from data (Wilkinson, 2012). Layering decomposes graphics into a small set of elements. For example, variables are translated into visual properties (axes, color, etc.) via 'aesthetic mapping', and geometric objects determine the shape of the graph itself. A graphic then is a combination of several elements, and layering allows for building complex figures. The user can add or change layers without breaking the entire figure, making the graphics transparent and manipulable.

`mice` imputation models are substantive model agnostic: variable-by-variable definition of imputation models. `ggmice` supports this. Because `mice` is substantive-model agnostic, `ggmice` offers tools to inspect whether the imputation model fits the observed data, instead of imposing particular analysis models. Assumptions: imputation model should fit the observed part, imputation model should be congenial `ggmice` provides tools to see observed and imputation side-by-side, adhering to advice to evaluate the distribution of observed and imputed data (Nguyen et al., 2017)

What does ggmice aim to solve?

`ggmice` offers a unified toolbox for the development and evaluation of imputation models. The `ggmice` package aims to solve the following problems:

- No direct graphical comparisons between incomplete and imputed data. Visualizations with `ggmice` can be made to inspect marginal and joint distributions of incomplete variables side-to-side to compare incomplete and imputed data. Without `ggmice`, it is cumbersome to see the data before and after imputation in comparable plots. `ggmice` offers functions that visualize incomplete data distributions and their completed counterparts.
- No previously existing methods for visualizing `mice` imputation models. The structure of `mice` imputation models (i.e., predictor matrix and methods vector) are often printed to the R console as text. Or, [experientially?], exported to spreadsheets. Complex imputation models are easier to review through visualization. `ggmice` provides imputation model plots that make complex imputation specifications easier to review and discuss.
- Limited chart types and design options for imputed data. Existing plotting tools for `mids` objects not easily editable or modifiable with different chart types. Visualizations with

ggmice can be extended and adapted with tidyverse functions to create publication-ready graphs of imputed data (reflecting the uncertainty due to missingness).

Existing solutions

The ggmice package fills a hiatus in the software landscape, with other packages generally supporting either mice or ggplot2, but not both. Table REF provides an overview of R packages that offer visualization tools for incomplete and imputed data. In short: with mice you cannot plot incomplete data distributions, all other packages lack support for mice-imputed data, and visualization of mice imputation models did not exist at all prior to ggmice.

In particular, visualization of mice imputation models did not exist prior to ggmice. And side-by-side visualizations of incomplete and imputed data was not supported in a unified graphical framework (nl., naniar's ggplot graphs with missing data, versus mice's lattice plots with imputed data).

Table 1: Differential functionality of ggmice versus other R packages

	ggmice	mice	naniar	VIM
Incomplete data (e.g., data.frame object)				
– missing data patterns	+	+	±	+
– joint & marginal distributions	+	−	+	+
Imputation model building				
– associations in observed data	+	−	−	−
– associations with missingness indicators	+	+	−	−
– conditional distribution with missingness indicator	+	−	−	−
– imputation models	+	−	−	−
Imputed data (e.g., mids object)				
– trace plot of the imputation algorithm	+	+	−	−
– joint & marginal distributions	+	+	−	−
User utility				
– extendability (i.e., non-deterministic graph types)	+	−	±	−
– editability (i.e., design adaptable for publication)	+	−	+	−
– popularity (i.e., monthly downloads)	−	+	±	±

Note. This is an incomplete overview of the R package's functionalities. The purpose of this table is to display *differences* in functionality based on the features offered by ggmice. I intend to show ggmice's added value compared to existing methodology, not a universal comparison. Symbols denote high/well supported (+), medium/moderate support (±), and low/not supported (−).

Limitations

`ggmice` does not pretend that visualization yields objective truth. A good graphical fit between observed and imputed data (e.g. overlapping scatterplots) is not a guarantee of statistical validity or correct missingness assumptions. Visual diagnostics can only suggest plausibility or highlight obvious misfits (imputation models should fit the observed part of the data). They cannot determinately identify the missing-data mechanism. The interpretation remains subjective (e.g. convergence, MAR assumption, etc.). Visualization is not a replacement for, but support to evaluation. Also check metrics such as FMI.

Future research & development

Planned extensions further align with the philosophy of making imputation workflows and their consequences visible. Some directions for future development include:

- Visualizing `mira` objects (analysis results across multiple imputations).
- Adding posterior predictive checks and over-imputation plots.
- Quantifying uncertainty at the pattern or cell level.
- Tools for sensitivity analyses (e.g. inducing MNAR structure in the observed data and inspecting effects on imputations and inferences).
- Investigating which data-level diagnostics are actually informative about imputation model adequacy in practice (i.e., when inferential validity cannot be evaluated).

In short, `ggmice` is an attempt to make imputation model assumptions and consequences as transparent as possible.

2 Chapter 2: evaluation

[TODO: link/insert + credit]

3 Chapter 3: convergence

[TODO: link/insert + credit]

4 Chapter 4: ggmice

[TODO: link/insert + credit]

5 Discussion

In this disseration, I have investigated how to evaluate imputation methodology. Instead of seeing imputation as a quick fix for treating missing data, I have approached it as a sociotachnical sysmtem: a methodological practice that requires continuous assessment and reflection. A central theme that runs through all chapters is that imputation methodology should be evaluated, whether computyationally or graphically. Evaluation is key both for methdolologists who develop imputation methodology, as well as for imputers who apply imputation in practice.

5.1 Main findings

The central theme of this dissertation is that imputation methodology requires verification and validation. Verification happens in simulation studies by means of computational evaluation of imputation methods. Validation happens in practice, by means of computational evaluation and graphical evaluation of the imputation workflows.

In chapter 2, we learnded that testing new imputation methods through simulation studies requires careful, standardized design. If this is not the case, simulation studies might not cover all relevant conditions, and results are not comparable between studies. To make imputation methods comparable across studies, we must standardize simulation designs so that different methods face similar, clearly documented scenarios, report design choices transparently, and publish research compendiums so that others can reproduce and extend the experiments. Even with such care, there is a limit to what simulation studies can tell us. We can know that a method worked under the scenarios we simulated, but we cannot be certain that it will work in practice. That is why we need validation.

In chapter 3 we studied the algorithmic convergence of the `mice` algorithm. Diagnosing non-convergence quantitatively is a form of computational evaluation of the imputation workflow. Since `mice` converges in distribution, and not to a point, non-convergence is hard to diagnose. We learned that commonly used metrics and thresholds for non-convergence in iterative algorithms such as `mice`, may not be directly applicable to FCS. Simulation results show that MICE reaches stable output before non-convergence metrics would indicate so. There is a mismatch between theoretical convergence criteria and practical algorithmic behavior. That is why we still have to rely on the current standard advised evaluation tool for algorithmic convergence: visual inspection. Inspecting traceplots for signs of non-convergence is a form of visual diagnostic evaluation of imputation workflows, or graphical evaluation. Graphical evaluation was the topic of the fourth chapter.

Visual inspection aids imputers in developing and evaluating imputation models, because formal tests are not available. The fifth chapter introduces the R package `ggmice`. `ggmice` offers tools for the graphical evaluation of the imputations models that were previously unavailable. `ggmice` functions aid imputers in visualizing incomplete data and imputed data side-by-side,

displaying missingness and imputation models in an interpretable way, and by producing plots that can be extended and customized for publication. These visual diagnostics support existing evaluations whenever quantitative metrics and substantive reasoning are unfeasible.

5.2 Recommendations

For colleagues who design and study imputation methods, this dissertation suggests the following:

- Design simulation studies in a structured and reproducible manner. Make sure that the simulation conditions and performance measures cover all relevant scenarios and are comparable across methods. Report all design choices, and publish code (and data if applicable) alongside methodological papers.
- Engage with applied researchers. Talk to your intended audience (i.e., applied researchers), or at least find out what they are struggling with via discussion forums or surveys. This should inform the design of methods and diagnostics.

For researchers who implement imputation in their own studies, the main advice can be summarized as:

- Think before you code. Do not treat `mice` or any other package as a black box that magically treats your missing data problem. Consider the missingness mechanisms and your analysis model(s) before specifying imputation methods and predictor sets. Your substantive knowledge is valueable for treating the missignness.
- Look at the data, plot the data. Inspect incomplete variables, missingness patterns, and imputed values using graphics. Use visualization to inform model building, diagnose misfit, and communicate uncertainty.
- Treat diagnostics as part of the analysis, not an afterthought. Convergence plots, distributional comparisons, and sensitivity analyses should be integrated into the workflow and reported.
- Please cite the software you use. For R packages, it's as easy as running `citation("ggmice")`.

5.3 Limitations

Computational and graphical evaluation are not a substitute for thinking. Simulation studies, diagnostic metrics and plots can offer detailed insight into imputation workflows, but they may also create a false sense of certainty. For example, imputed values that lie within the range of the observed data may give a false sense of certainty against a MNAR mechanism, which is an untestable assumption. No amount of computation or visualization can prove that the missing-data mechanism is correctly assumed. Instead, one should use computational evaluation as complement to (not a replacement for) substantive knowledge and expert insight.

This dissertation investiagtes imputation workflows at the methodological level, without systematic user feedback. I have not tested the evaulation tools I propose experimentally. Assessing their causal impact on user behavior and imputation quality in practice was not feasible. I did receive feedback via teaching, workshops, collaborations, and online forums GitHub and StackOverflow. This informal evaluation is an indication of user appreciation(?). And I have used this input for example to further develop `ggmice`. So even though it might be impossibel to draw evidence-based conclusions about the efficacy of the tools I developed, we may still call the development evidence-informed, maybe?

Future direction: automated sensitivity analyses and over-imputation. Sensitivity analyses that vary model assumptions, include alternative predictors, or impose plausible MNAR structures can reveal how fragile conclusions are to changes in imputation choices. Over-imputation can reveal imputation model misfit on observed part of the data. Since publishing, there has not been a pre-reg format

Since this dissertation relies on the R language and centers the `mice` package, it is not certain how results translate to the wider imputation ecosystem. Concepts may generalize beyond `mice` in R. The focus reflects the current dominance of R and `mice` in many applied fields, and allows for depth and specificity in tool development. While this set-up is popular, the data science landscape is expanding, with many machine learning and deep learning methods emerging as candidate imputation engines. These methods are not (yet) widely implemented within `mice`. They might be able to better approximate the posterior predictive distribution of the missing values than `mice` methods, but this requires dedicated research and evaluation, which is outside of the scope of this dissertation.

Some other directions for future research include computational evaluation of variable importance within imputation models, integration of `ggmice` with `naniar`, and better support for non-numeric variables. The visualizations implemented in `ggmice` are generally better suited for numeric data, as opposed to categorical variables (or even open text field data, which is not supported by `mice` currently). Future research may focus on associations of categorical variables with missingness indicators, and convergence parameter other than SD of imputed values.

5.4 Outlook

Missingness will be a persistent problem in research data. There is no foreseeable future in which all empirical datasets are complete. I therefore expect imputation methodology to conBut the way we deal with data might change. But the way we deal with data might change.

With the rise of machine learning and deep learning methods (or "AI"), evaluation will become ever more important. These new methods will also require new verification and validation tools, especially if the framework for intended applications changes. Many novel methods are designed under a prediction framework, not a statistical inference framework. Instead of inferring from a sample to the population, the aim becomes to infer about an

individual case in the fututre. Methods intended for prediction may excel at point prediction, but ignore uncertainty in estimates. Omitting sampling variance from the estimates and to failing to incorporate between-imputation variance underrepresents the uncertainty. When such methods are used naively for inference, they produce confidence intervals that are too narrow and p-values that are too small. The result is an increased risk of spurious findings. Therefore, all models must propagate uncertainty.

Any model used in scientific inference should propagate uncertainty. This includes (but is not limited to) integrating missing-data uncertainty into model outputs and reporting interval estimates that align with the uncertainty in the data and the modeling process. This also holds for graphical presentations.

Trust in science requires an honest reflection of uncertainty in our analyses. So please be open about uncertainty and limitations of scientific studies. And while we're at it, also please publish null results (e.g., via registered reports or pre-registrations), share data and code with other scientists and the public, and publish open access (including educational materials). At the same time, we should change the way we evaluate science. This dissertation is an example of the new 'recognition and rewards' vision. It includes software as an actual chapter, not something extra. This is part of the current movement of valuing diverse forms of contribution beyond traditional papers. I hope this can serve as an example for other PhD candidates and researchers to reflect on what they consider scientific output. I hope and to claim space for software, open data, and methodological infrastructure as central achievements. I hope to inspire others to claim space for 'alternative' output such as software and data as full-fledged research output.

Finally, it is important to note that imputation is not always the right response to missingness. Missing data can signal a mismatch between our data collection methods and the lived experiences of participants. Systematic patterns of non-response may point to issues in the survey design, accessibility, or trust. In such cases, treating missingness solely as a statistical nuisance to be 'fixed' risks overlooking important social and ethical dimensions of research, especially in the social and biomedical sciences, where rows in the data represent human beings. I would love to study missingness as a method to tell you about mismatches between data collection and participant perspectives. In my own life, I have experienced … [TODO: positionality??]

In short, I hope this dissertation encourages to question defaults, to make evaluation practices explicit, and to contribute to a scientific culture that takes uncertainty—and our responsibilities in representing it—seriously.

References

Allison, P. D. (2002). *Missing Data*. SAGE Publications, Inc. https://doi.org/10.4135/9781412985079

Alsalti, T., Rohrer, J. M., & Arslan, R. C. (2026). Thinking Clearly About Sampling and Representation With the Total Survey Error Framework. *Social and Personality Psychology Compass*, *20*(7), e70160. https://doi.org/10.1111/spc3.70160

Bartlett, J. W., Seaman, S. R., White, I. R., & Carpenter, J. R. (2015). Multiple imputation of covariates by fully conditional specification: Accommodating the substantive model. *Statistical Methods in Medical Research*, *24*(4), 462–487. https://doi.org/10.1177/0962280214521348

Carpenter, J. R., & Smuk, M. (2021). Missing data: A statistical framework for practice. *Biometrical Journal*, *63*(5), 915–947. https://doi.org/10.1002/bimj.202000196

Curzon, P., Dorling, M., Ng, T., Selby, C., & Woollard, J. (2014). *Developing computational thinking in the classroom: A framework*. Computing at School.

de Ruiter, J. P., & Albert, S. (2017). An Appeal for a Methodological Fusion of Conversation Analysis and Experimental Psychology. *Research on Language and Social Interaction*, *50*(1), 90–107. https://doi.org/10.1080/08351813.2017.1262050

Deming, W. E. (1944). On Errors in Surveys. *American Sociological Review*, *9*(4), 359–369. https://doi.org/10.2307/2085979

Dudek, G. (2004). Computational Evaluation. In G. Dudek (Ed.), *Collaborative Planning in Supply Chains: A Negotiation-Based Approach* (pp. 165–213). Springer. https://doi.org/10.1007/978-3-662-05443-7_7

Evaluation measures (information retrieval). (2025). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Evaluation_measures_(information_retrieval)&oldid=1329650960

Federal Committee on Statistical Methodology. (2001). *Statistical Policy Working Paper 31. Measuring and Reporting Sources of Error in Surveys*. https://doi.org/10.21949/1529874

Gelman, A., & Shalizi, C. R. (2013). Philosophy and the practice of Bayesian statistics. *British Journal of Mathematical and Statistical Psychology*, *66*(1), 8–38. https://doi.org/10.1111/j.2044-8317.2011.02037.x

Graham, J. W. (2012). *Missing Data*. Springer. https://doi.org/10.1007/978-1-4614-4018-5

Hand, D. (2020). *Dark data: Why what you don't know matters*. Princeton University Press.

Hand, D. (2025). Statistics: An Overview. In *International Encyclopedia of Statistical Science* (pp. 2654–2659). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-662-69359-9_649

Little, R. J. A., & Rubin, D. B. (2019). *Statistical analysis with missing data. Third edition*. John Wiley & Sons, Ltd. https://doi.org/10.1002/9781119482260

Little, & Rubin. (2020). *Statistical Analysis with Missing Data, Third Edition | Wiley Series in Probability and Statistics*. https://onlinelibrary.wiley.com/doi/book/10.1002/9781119482260

Liu, D., Oberman, H. I., Muñoz, J., Hoogland, J., & Debray, T. P. A. (2023). Quality Control, Data Cleaning, Imputation. In F. W. Asselbergs, S. Denaxas, D. L. Oberski, & J. H. Moore (Eds.), *Clinical Applications of Artificial Intelligence in Real-World Data* (pp. 7–36).

Springer International Publishing. https://doi.org/10.1007/978-3-031-36678-9_2
Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. https://doi.org/10.1017/CBO9780511809071
Molenberghs, G., Fitzmaurice, G., Kenward, M. G., Tsiatis, A., & Verbeke, G. (Eds.). (2014). *Handbook of Missing Data Methodology*. Chapman and Hall/CRC. https://doi.org/10.1201/b17622
Neyman, J. (1934). On the Two Different Aspects of the Representative Method: The Method of Stratified Sampling and the Method of Purposive Selection. *Journal of the Royal Statistical Society*, *97*(4), 558–625. https://doi.org/10.2307/2342192
Nguyen, C. D., Carlin, J. B., & Lee, K. J. (2017). Model checking in multiple imputation: An overview and case study. *Emerging Themes in Epidemiology*, *14*(1), 8. https://doi.org/10.1186/s12982-017-0062-6
Nguyen, C. D., Carlin, J. B., & Lee, K. J. (2021). Practical strategies for handling breakdown of multiple imputation procedures. *Emerging Themes in Epidemiology*, *18*, 5. https://doi.org/10.1186/s12982-021-00095-3
Oberman, H. I. (2022). *Ggmice: Visualizations for 'mice' with 'Ggplot2'* [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.6532702
Oberman, H. I., University, U., Utrecht, U. M. C., Volker, T., Vink, G., Vink, P., Wallis, J., & Lang, K. (2025). *Ggmice: Visualizations for 'mice' with 'Ggplot2'* (Version 0.1.1) [Computer software]. https://cran.r-project.org/web/packages/ggmice/index.html
Rosenberg, A. (2018). *Philosophy of Social Science*. Routledge. https://books.google.com?id=GRdWDwAAQBAJ
Rubin, D. B. (1976). Inference and Missing Data. *Biometrika*, *63*(3), 581–592. https://doi.org/10.2307/2335739
Rubin, D. B. (1987). *Multiple Imputation for nonresponse in surveys*. Wiley.
Rubin, D. B. (1996). Multiple Imputation after 18+ Years. *Journal of the American Statistical Association*, *91*(434), 473–489. https://doi.org/10.1080/01621459.1996.10476908
Schafer, J. L. (1997). *Analysis of Incomplete Multivariate Data*. Chapman and Hall/CRC. https://doi.org/10.1201/9781439821862
Schafer, J. L., & Graham, J. W. (2002). Missing data: Our view of the state of the art. *Psychological Methods*, *7*(2), 147–177. https://www.ncbi.nlm.nih.gov/pubmed/12090408
Sommerville, I. (2015). *Software engineering* (Tenth edition). Pearson.
van Buuren, S. (2007). Multiple imputation of discrete and continuous data by fully conditional specification. *Statistical Methods in Medical Research*, *16*(3), 219–242. https://doi.org/10.1177/0962280206074463
van Buuren, S. (2018). *Flexible imputation of missing data* (2nd ed.). CRC Press.
van Buuren, S., Brand, J. P. L., Groothuis-Oudshoorn, C. G. M., & Rubin, D. B. (2006). Fully conditional specification in multivariate imputation. *Journal of Statistical Computation and Simulation*, *76*(12), 1049–1064. https://doi.org/10.1080/10629360600810434
Wilkinson, L. (2012). The Grammar of Graphics. In *Handbook of Computational Statistics* (pp. 375–414). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-21551-3_13

CV

[TODO: write]